

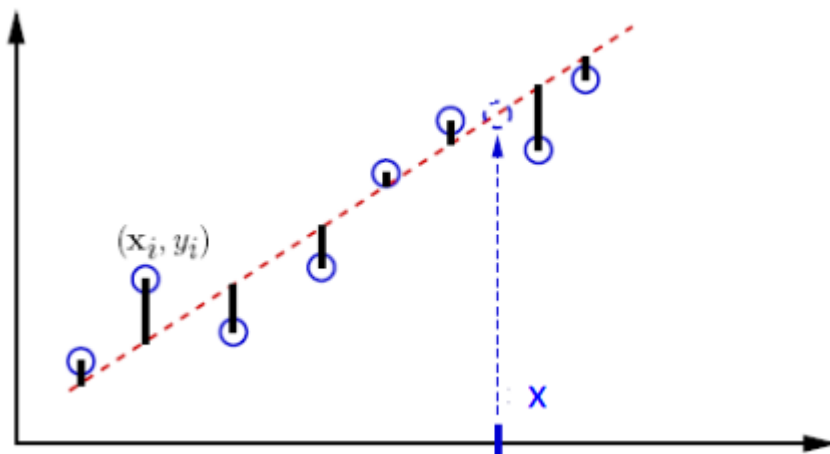
Linear Regression: Basis Functions

Linear Regression

- Hypothesis

$$y = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d = \sum_{j=0}^d \theta_j x_j$$

- Fit a line by minimizing sum of squared errors.

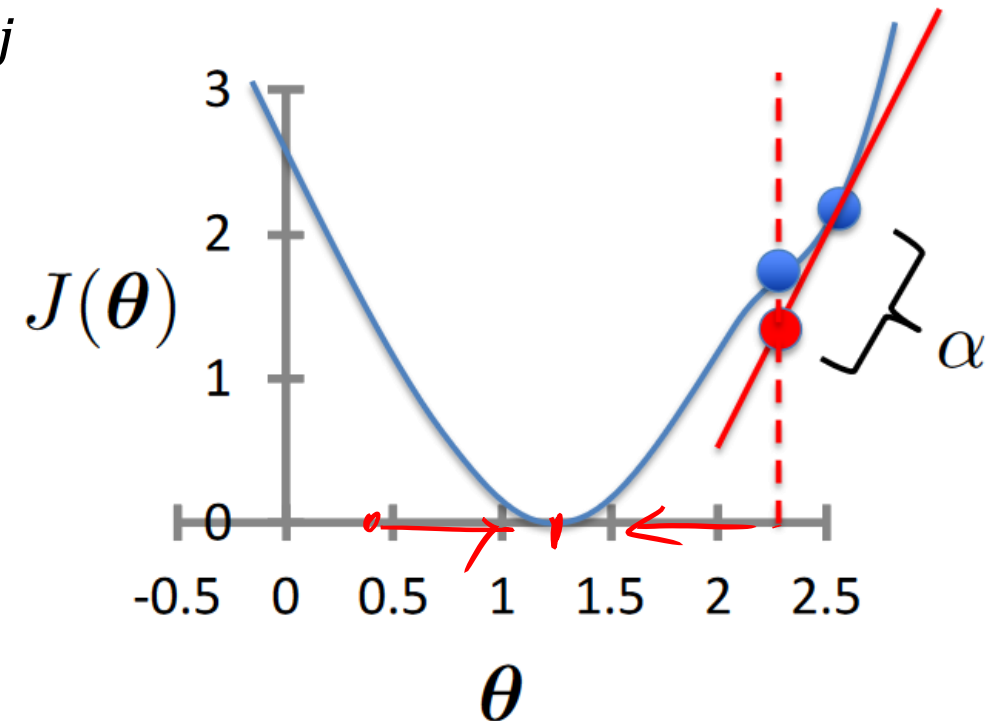


$\min$

Gradient Descent

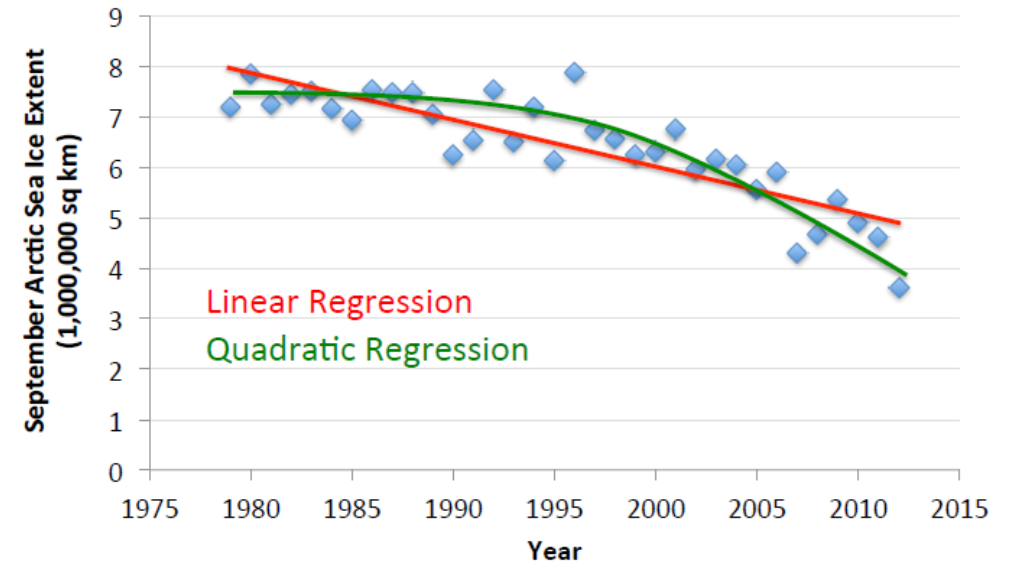
- Initialize θ
- Repeat until convergence (simultaneous update for $j = 0, 1, \dots, d$)

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$



Regression

- Given: Labelled training data $\{(x^{(1)}, y^{(1)}), (x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$
 - Data $\mathbf{X} = \{x^{(1)}, \dots, x^{(n)}\}$ where $x^{(i)} \in \mathbb{R}^d$
 - Labels $\mathbf{y} = \{y^{(1)}, \dots, y^{(n)}\}$ where $y^{(i)} \in \mathbb{R}$



Extension of Linear Regression: More Complex Models

- The inputs x for linear regression can be of different forms.
 - Original inputs
 - ⇒ • Transformations of inputs (e.g. log, exp, etc.)
 - • Polynomial transformation
 - • Basis expansions
 - • Interactions between variables (e.g. $x_3 = x_1 \cdot x_2$)
 - Etc.
- This enables us to use linear regression techniques to fit non-linear datasets.

Linear Basis Function

- Generally,

$$h_{\theta}(\mathbf{x}) = \sum_{j=0}^d \theta_j \underline{\varphi_j(\mathbf{x})}$$

- $\varphi_0(\mathbf{x}) = 1$ enables θ_0 as a bias

φ_j

- The linear basis function is the simplest case

$$\underline{\varphi_j(\mathbf{x}) = x_j}$$

Other Basis Functions

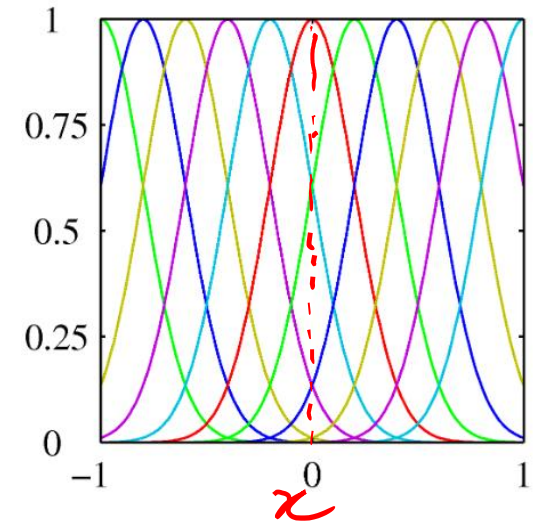
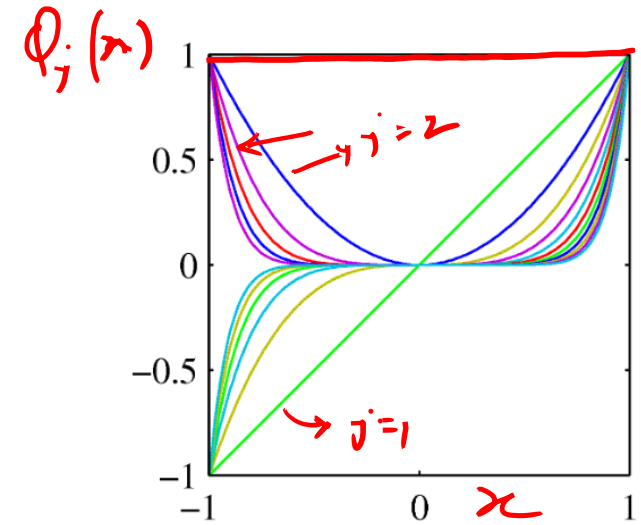
- Polynomial basis functions:

$$\varphi_j(x) = x^j$$

- These are global
 - A small change in x affects all basis functions
- Gaussian basis functions:

$$\varphi_j(x) = \exp \left\{ -\frac{(x - \mu_j)^2}{2s^2} \right\}$$

- These are local
- A small change in x only affect nearby basis functions
- μ_j control location
- s control scale (width)



Other Basis Functions (contd.)

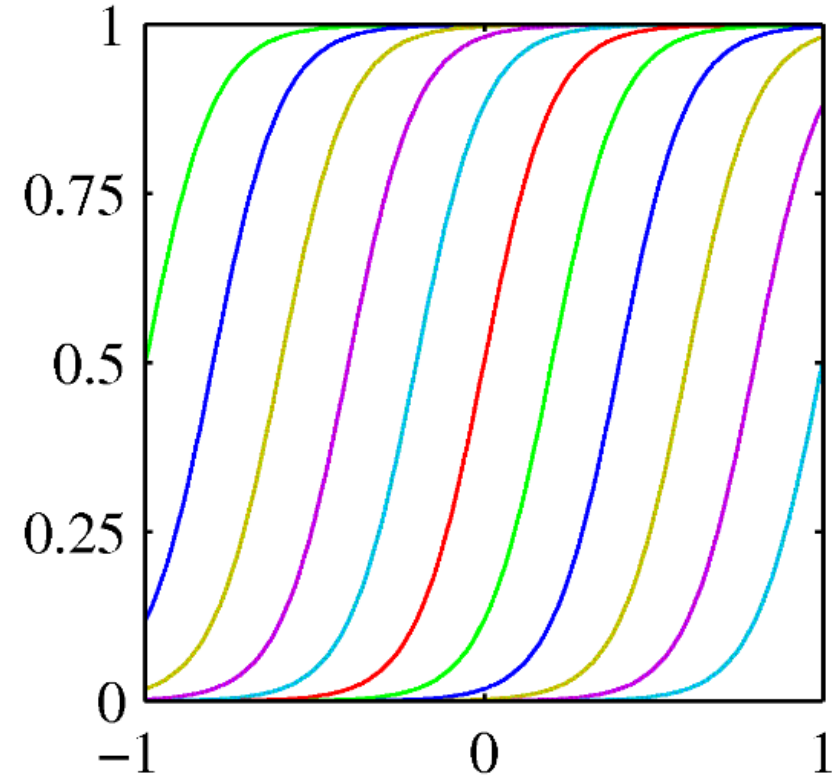
- Sigmoidal basis functions:

$$\varphi_j(x) = \sigma\left(\frac{x - \mu_j}{s}\right)$$

where

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

- These are also local (*Transition is local*)
- μ_j control location
- s control scale (slope)



* For the j th polynomial basis fn $\phi_j(x) = x^j$ $\frac{d \phi_j(x)}{dx} = j x^{j-1}$

$x \rightarrow x + \delta$

$$(x + \delta)^j = x^j + j x^{j-1} \delta + O(\delta^2)$$

$$\underline{\underline{\Delta \phi_j}} = (x + \delta)^j - x^j \approx \underline{\underline{j x^{j-1} \delta}} = \phi_j'(x) \delta$$

A small change in x affects all ϕ_j . Hence it is global

* For Gaussian $\phi_j(x) = \exp\left[-\frac{(x - \mu_j)^2}{2\sigma^2}\right]$

$$\Delta \phi_j \approx \phi_j'(x) \delta$$

If x is far from μ_j then $\phi_j(x) \approx 0$ and its derivative is also small. Therefore nearby basis fn changes.

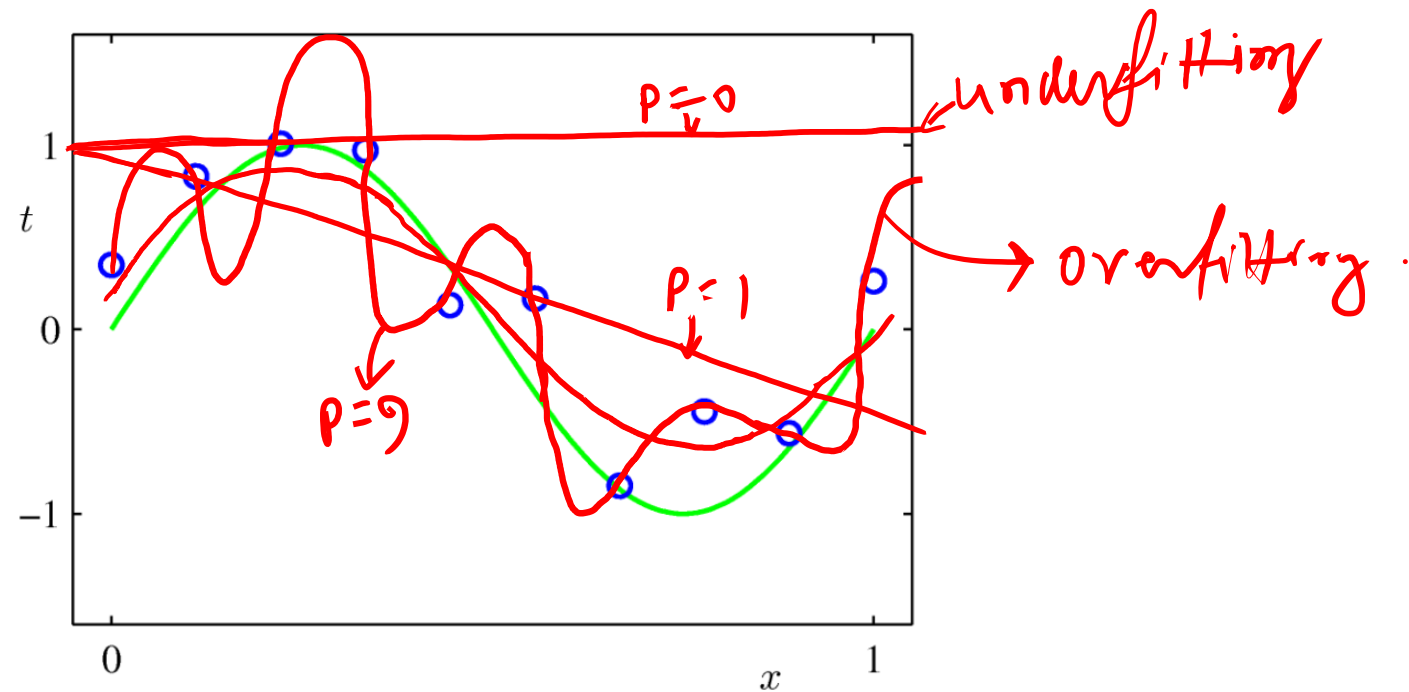
Choice of Basis Functions

Data Property	Basis
Smooth, global trend	Polynomial
Local nonlinearities	Gaussian
Periodic	Fourier
Localized-frequency changes	Wavelets
Saturation effect/monotonic transitions	Sigmoid
Very Complex	Neural Networks

Fitting Polynomial Curve with a Linear Model

$$y = \theta_0 x^0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_p x^p = \sum_{j=0}^p \theta_j x^j$$

$t = \sin \pi x + \epsilon$
 $x \in [0, \dots, 9]$
 $\epsilon \sim \mathcal{N}(0, 9)$



Linear Basis Function Models

- Basic linear model:

$$h_{\theta}(\mathbf{x}) = \sum_{j=0}^d \theta_j x_j$$

- General Linear model:

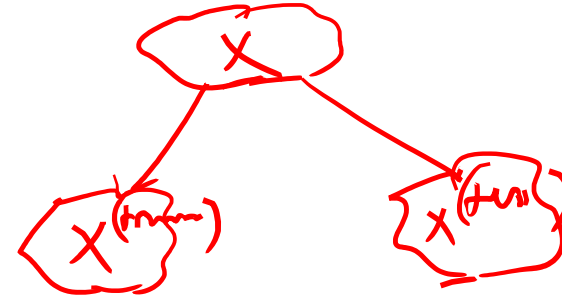
$$h_{\theta}(\mathbf{x}) = \sum_{j=0}^d \theta_j \varphi_j(\mathbf{x})$$

- Once, the data is updated by the outputs of the basis functions, fitting the generalized model is exactly the same problem as fitting the basic model.

The central challenge of ML

- The model must perform well on new, previously unseen inputs
- Not just those, on which the model has been trained
- Ability to perform well on previously unobserved data is called generalization.

Generalization Error



- When training a machine learning model we have access to a training set
 - We can compute some error measure on the training set, called training error
- ML differs from optimization
 - Generalization error (also called test error) to be low
- Generalization error: Expected value of the error on a new input
 - Expected value is computed as average over inputs taken from distribution encountered in practice
 - In linear regression with squared error criterion the expected error is

$$\frac{1}{2n^{(test)}} \| \mathbf{X}^{(test)} \boldsymbol{\theta} - \mathbf{y}^{(test)} \|_2^2$$

Estimating the generalization error

- We estimate generalization error of a ML model by measuring performance on a test set that were collected separately from the training set
- In linear regression example we train model by minimizing the training error $\frac{1}{2n^{(train)}} \|\mathbf{X}^{(train)} \boldsymbol{\theta} - \mathbf{y}^{(train)}\|_2^2$
- However, we actually care about the test error
 $\frac{1}{2n^{(test)}} \|\mathbf{X}^{(test)} \boldsymbol{\theta} - \mathbf{y}^{(test)}\|_2^2$
- When we observe train set only during training, how do we deal with test case?

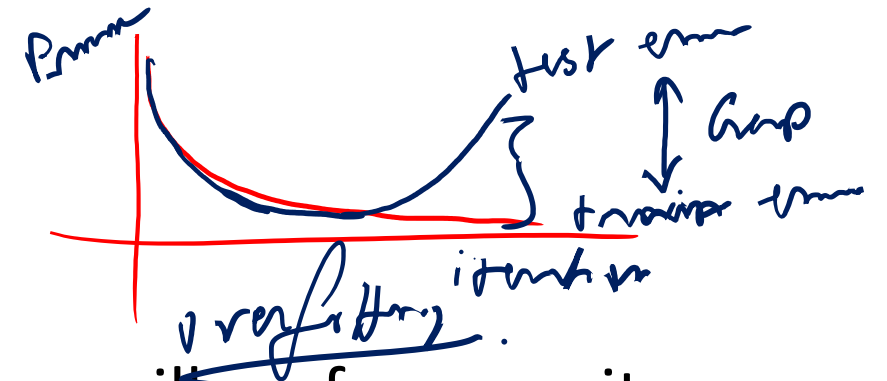
Statistical Learning Theory

- Need assumptions about training and test sets
 - Training/test data arise from same process
 - We make use of i.i.d. assumptions
 - Examples in each data set are independent
 - Training set and testing set are identically distributed
 - We call the shared distribution, the data generating distribution p_{data}
- Probabilistic framework and i.i.d. assumption allows us to study relationship between training and testing error

Why are training/test errors unequal?

- Expected training error of a randomly selected model is equal to expected test error of model
 - If we have a joint distribution $p(x,y)$ and we randomly sample from it to generate the training and test sets. For some fixed value θ the expected training set error is the same as the expected test set error
- But we don't fix the parameters θ in advance!
 - We sample the training set and then use it to choose θ to reduce training set error

Under- and Over-fitting



- Factors determining how well an ML algorithm will perform are its ability to:
 - Make the training error small
 - Make gap between training and test errors small
- They correspond to two ML challenges
 - Underfitting: Inability to obtain low enough error rate on the training set
 - Overfitting: Gap between training error and testing error is too large
- We can control whether a model is more likely to overfit or underfit by altering its capacity

